LABORATORY ACTIVITY: POCKET NOTES

An app that remembers things, tested on your own device

Build a small app that remembers things after it's closed. Then try to break it on purpose, and prove everything with screenshots from your own device. AI can write the code for you. It can't run the app on your device or explain what happened. That part is yours.

Ground rules

- **AI is allowed.** Use any tool you like. You just have to disclose every prompt (Part 6).
- **Your device, your app.** Every screenshot comes from your own phone or emulator running your own app.
- **Circle the proof.** Draw a circle, arrow, or label on what each screenshot proves. An unmarked screenshot doesn't count.
- **Your name goes in your data.** The first record you add must contain your name.

Part 1 — Your record type

Look at the last digit of your class number. That's the thing your app stores. Every record also gets an auto-increment `id`, and your table is named after your record type (like `plants`).

Last digit	Record type	Fields (besides id)
0	Songs	title, artist
1	Expenses	item, amount
2	Tasks	task, due date
3	Plants	plant name, watering notes
4	Contacts	name, phone number
5	Gym log	exercise, reps
6	Recipes	dish, ingredients
7	Books	title, author
8	Movies	title, rating
9	Class schedule	subject, room

Part 2 — Build it

Plain UI is fine: one screen, a list, an add/edit dialog, and a theme switch. Setup: `flutter pub add shared_preferences sqflite path`

Your app must have:

1. A **light/dark theme switch**. The choice is saved with `SharedPreferencesAsync` and restored when the app launches.
2. A **list of your records**, loaded from a sqflite table when the app launches.
3. **Add, edit, and delete** for a record.
4. The database **opened once**, with that one instance reused everywhere.
5. Every update and delete using `where` + `whereArgs`, with no string-gluing into SQL.

Part 3 — Test it on your own device (32 pts)

Force-kill means swipe the app away from your recent-apps screen, then open it again from the home screen or app drawer. Hot reload, hot restart, and stopping from the IDE don't count. Capture all 8:

#	Capture	Must show	Done
1	Theme, both modes	Light theme and dark theme, each on screen. Circle the switch.	[0]
2	Theme survives	Set dark. Screenshot before the force-kill, and again after reopening. Circle the theme on both.	[0]
3	Records, before	Add 3 records. The first must contain your name. Screenshot the list.	[0]
4	Records, after	Force-kill, reopen, and screenshot the same list. Circle the record with your name.	[0]
5	Edit	Before and after editing one record. Circle the change.	[0]
6	Delete	Before and after deleting one record. Circle what's gone.	[ ]
7	Database proof	The rows in your database, next to the app's list. They must match. Use Android Studio's Database Inspector (App Inspection), or print `await db.query('your_table')` to the console.	[ ]
8	whereArgs	Your delete code, with `where` and `whereArgs` circled.	[ ]

Part 4 — Break it on purpose (16 pts)

Good devs try to break their app before users do. Two experiments. Each one needs screenshots and a short written answer.

Experiment 1: The wrong-theme flash

1. Set the dark theme, then force-kill the app.
2. Change your startup so the first screen is drawn **before** the saved theme is read. Don't `await` the prefs read before `runApp()`. Load it in `initState` and call `setState` when it arrives.
3. Cold start the app from the home screen. Screenshot the wrong-theme frame before it flips. Can't catch it? Wrap the read in `Future.delayed(const Duration(seconds: 2), ...)`, or pause a screen recording.
4. Put your fix back.

Write it: What did you see, and why did it happen? Name the cause.

It had a delay for a few minutes after I change it back to its state. The cause was the last minute editing of the code when I ran it.

Experiment 2: What survives a restart?

1. Add one thing you **don't** save: a "favorites only" chip, a counter, or a search box held in a plain `State` variable.
2. Switch it on, with the dark theme and your records showing. Screenshot.
3. Force-kill, reopen, and screenshot again.

Write it: What survived? What didn't? Explain why, in terms of where each one lives.

Part 5 — Storage justification (12 pts)

Answer for **your own app** and your own record type, about two sentences each. Generic answers don't count.

a. Why does the theme belong in SharedPreferences?

b. Why do your records belong in the database?

c. What breaks if you put your records in SharedPreferences instead? Be specific to your record type.

Part 6 — AI prompt disclosure and log (20 pts)

AI is allowed. Hiding it isn't. List every tool and every prompt, word for word and in order. No summaries.

Tool(s) used: __Claude ai_____

#	Your prompt (word for word)	What the AI gave you	Accepted, changed, or rejected? What did you change?
1	Can you edit the code? (screenshot or code sending)	The new improve code	Accepted
2	Sending a screenshot then Ai scans the screenshot	Provided a solution	Accepted
3			
4			

Need more rows? Add them, or paste screenshots of your chat after this table.

One thing the AI got wrong, or that you had to fix:

When Ai send me the full code, there are errors within the context section. I had to fix it by editing it or prompt it back to an ai

No AI used? Write "No AI used" and your typed name below. You get full credit for this part.

I tested this app on my own device, and the screenshots are mine.

Typed name: _____ Date: _____

What a strong prompt includes

- **Context:** say it's Flutter, name the packages, and say what your app and record type are.
- **Specific task and constraints:** spell out exactly what you need, like "use `SharedPreferencesAsync`" or "use `where` + `whereArgs`."
- **Output format:** ask for a full file with comments, or step-by-step instructions.
- **Iteration:** when something breaks, paste the error or describe what you saw, and ask for a fix.
- **Verification:** ask it to explain why something works, or check its answer against your own test.

Weak: `make notes app`

Strong: "I'm building a Flutter app called [name] that saves [your records] with sqflite. Write a [Record] model with toMap and fromMap, plus one function that opens the database once. Use where + whereArgs for update and delete. Give me the full file with comments, and explain why the database is only opened once."

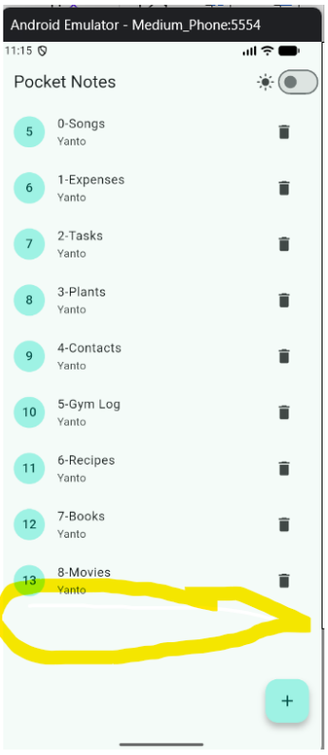
How you're graded (80 pts)

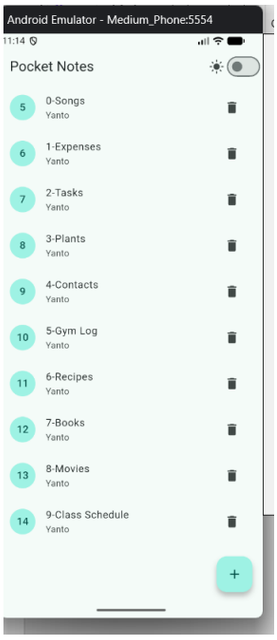
Criterion	Pts	Where	Based on
Working App Evidence	32	Part 3	How many of the 8 screenshots are present and marked
Break-It Experiments	16	Part 4	Screenshot, cause, before/after, and explanation (4 components)
Storage Justification	12	Part 5	Parts a, b, and c, specific to your own app
AI Use and Prompt Quality	20	Part 6	Full disclosure (8) and the 5 prompt qualities above (12)
Total	**80**		

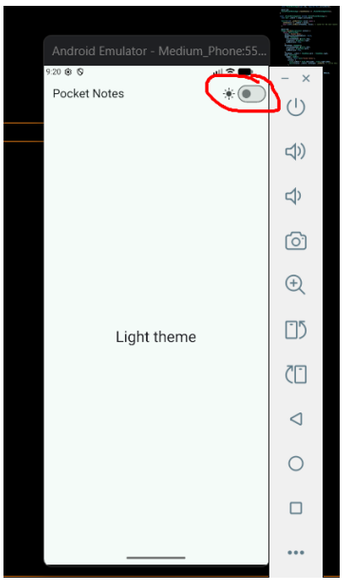
Submit

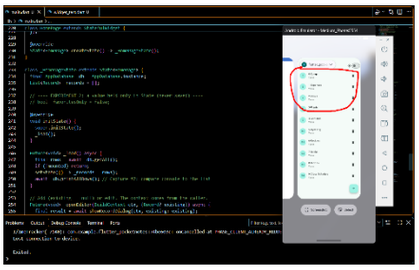
Submit in Google Classroom before the deadline posted there.

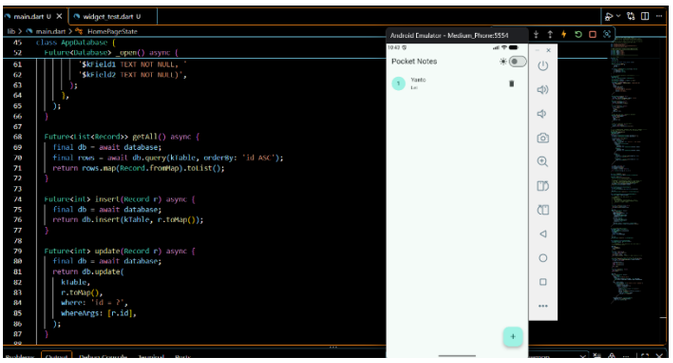
- **One PDF** named `CC105_PocketNotes_Surname_Firstname.pdf`, in this order: Part 3 screenshots (1 to 8), Part 4 screenshots and answers, Part 5 answers, Part 6 log.
- **Your project's repository in github. No Github, No Grade**











That's it. Build it, break it, prove it.